

Trabalho 06 - Redes Recorrentes

1st Nycksandro Lima dos Santos
Instituto de Computação
Universidade Federal do Amazonas
Manaus, Brasil
nycksandro.santos@icomp.ufam.edu.br

I. OBJETIVO

Esse trabalho tem por objetivo a identificação de atividades a partir dos sinais x, y e z de acelerômetros, contidos no dispositivo móvel Samsung Galaxy Nexus I9250 com o sistema operacional Android versão 5.1.1, equipado com o sensor de aceleração Bosh BMA220. Esses dados foram coletados a partir da base de dados UniMibSHAR [1] e contém informações de aceleração das seguintes atividades:

- 1) Levantando a partir de um estado sentado numa cadeira - “StandingUpFS”
- 2) Levantando a partir de um estado deitado na cama - “StandingUpFL”
- 3) Caminhando - “Walking”
- 4) Correndo - “Running”
- 5) Descendo uma escada - “GoingUpS”
- 6) Pulando - “Jumping”
- 7) Subindo uma escada - “GoingDownS”
- 8) Deitando numa cama a partir de um estado em pé - “LyingDownFS”
- 9) Sentando numa cadeira estando em pé - “SittingDown”

II. BASE DE DADOS

A base de dados os UniMibSHA [1] é composta por dois arquivos:

- 1) dados.csv
- 2) labels.csv

O arquivo dados.csv contém 1724 linhas contendo informações dos acelerômetros. Cada linha contém 453 valores tipo “double” dos acelerômetros x, y e z relativos a uma atividade. Os primeiros 151 valores correspondem a dados do acelerômetro x, os 151 seguintes, a dados do acelerômetro y e os últimos 151 valores, a dados do acelerômetro z.

O arquivo labels.csv contém também 1724 linhas com informações das atividades realizadas. Cada linha contém um número inteiro entre 1 e 9, correspondendo as atividades mostradas anteriormente. Existe uma correspondência entre uma linha do arquivo labels.csv e uma linha do arquivo dados.csv.

A. Adaptações

- Transformação para formato de sequência temporal: Cada amostra virou uma sequência temporal onde cada tempo tem 3 features (x, y, z).
- Ajuste dos rótulos: Originalmente eram de 1-9 e foram convertidos para 0-8.

- One-Hot Encoding das classes: Dessa forma, o número 3 $\rightarrow [0,0,0,1,0,0,0,0,0]$.
- Estruturação para LSTM: Foi necessário reorganizar os dados no formato exigido pela LSTM (samples, timesteps, features)

B. Divisão da Base de Dados

O Dataset [1] foi particionado aleatoriamente em 85% dos elementos para a partição de Treino, enquanto os outros 15% foram para partição de Validação.

III. ARQUITETURA DA REDE

O modelo possui a seguinte arquitetura:

- 1) Entrada (151, 3)
- 2) LSTM (64)
- 3) Dropout (0.3)
- 4) Dense (32, ReLU)
- 5) Dense (9, Softmax)
- 6) Saída (classe da atividade)

A arquitetura pode ser ilustrada pela figura a seguir:

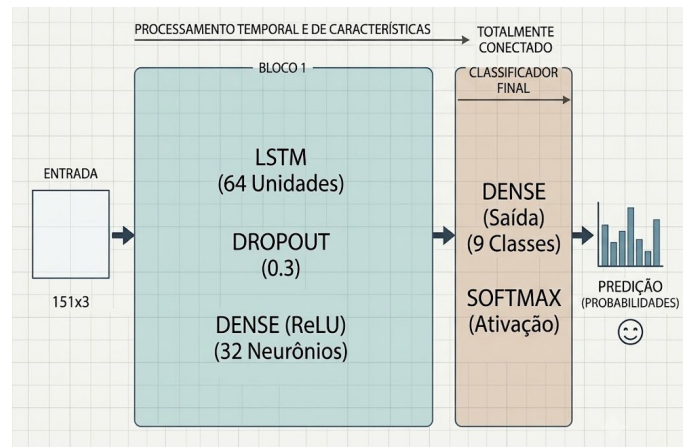


Fig. 1: Diagrama da Arquitetura do Modelo

IV. HIPERPARÂMETROS

O modelo foi configurado com os hiperparâmetros a seguir:

- Learning rate: 1×10^{-4}
- Batch size: 16
- Epochs: 50
- Optimizer: Adam
- Loss = categorical_crossentropy

V. CÓDIGO

A simulação foi realizada utilizando o seguinte código (versão simplificada) em Python. Os detalhes da implementação estão disponíveis em: Nycksandro_Lima_Trabalho06.

```
#Divisão dos dados
# Conjunto de treinamento: 85% dos dados e validação: 15% dos dados

X_train, X_val, y_train, y_val = train_test_split(
    X,
    y_cat,
    test_size=0.15,
    random_state=42,
    stratify=y)

# Carregando o modelo

model = Sequential()
model.add(
    LSTM(units=64, input_shape=(151, 3)))
model.add(Dropout(0.3))
model.add(Dense(32, activation='relu'))
model.add(Dense(9, activation='softmax'))
model.summary()

# Compilação
model.compile(
    optimizer= Adam(learning_rate=0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy'])

# Treinamento do modelo

history = model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=16,
    validation_data=(X_val, y_val),
    verbose=1)

# Avaliação do modelo

loss, acc = model.evaluate(X_val, y_val)
```

VI. RESULTADOS

Ao fim do treinamento na época 50, foi realizado a validação no modelo no conjunto de validação e os resultados obtidos foram:

- Loss: 0.198
- Acurácia: 0.923

A. Acurácia

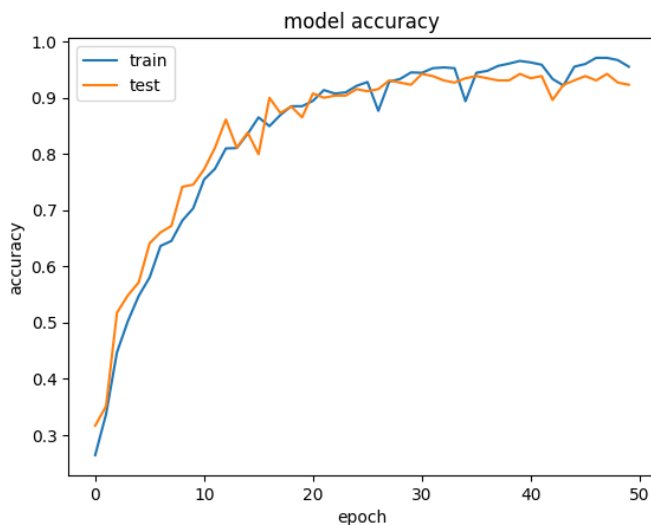


Fig. 2: Acurácia do Modelo ao longo do Treinamento

O gráfico de acurácia demonstra um processo de aprendizado altamente estável e bem-sucedido, onde a curva do conjunto de teste acompanha de perto a evolução do treinamento, atingindo um platô estável por volta da época 30. A convergência paralela entre as duas linhas descarta cenários de underfitting inicial e comprova a eficácia da camada de Dropout (0.3) em mitigar o overfitting na rede LSTM, garantindo que o modelo consolidasse seu aprendizado de forma robusta e equilibrada.

B. Perda (Loss)

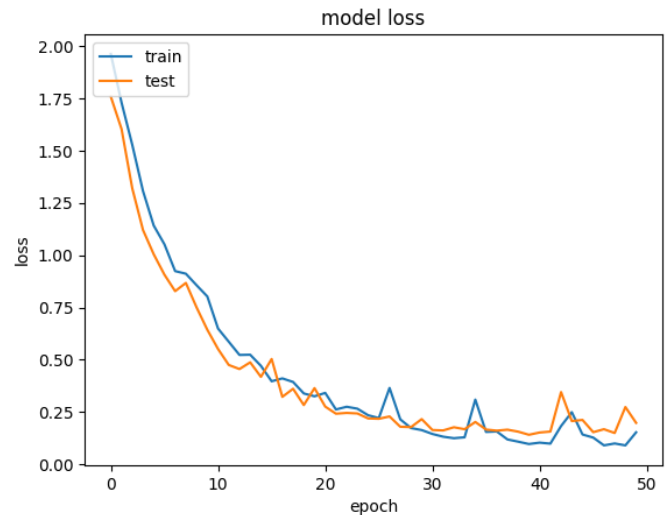


Fig. 3: Perda do Modelo ao longo do Treinamento

A curva de perda (loss) valida o comportamento seguro da otimização, apresentando um decréscimo acentuado e contínuo do erro tanto para os dados de treino quanto de teste nas primeiras 20 épocas. A estabilização das curvas no chão do gráfico em um valor mínimo (0,198) indica que os pesos da camada LSTM e das densas foram ajustados de maneira ideal pelo algoritmo de otimização, sem apresentar qualquer sinal de divergência ou desgaste computacional tardio.

VII. MATRIZ DE CONFUSÃO

A matriz de confusão evidencia visualmente o impacto de um severo desbalanceamento de classes no dataset, revelando que a rede foi induzida a priorizar o aprendizado das atividades dinâmicas devido à abundância de dados dessas categorias. Enquanto as classes com maior suporte geram uma linha diagonal limpa e precisa, os quadrantes correspondentes às classes estáticas e de transição mostram uma distribuição dispersa de erros, evidenciando que o modelo frequentemente confunde esses estados sutis por falta de volume amostral para mapear suas fronteiras de decisão.

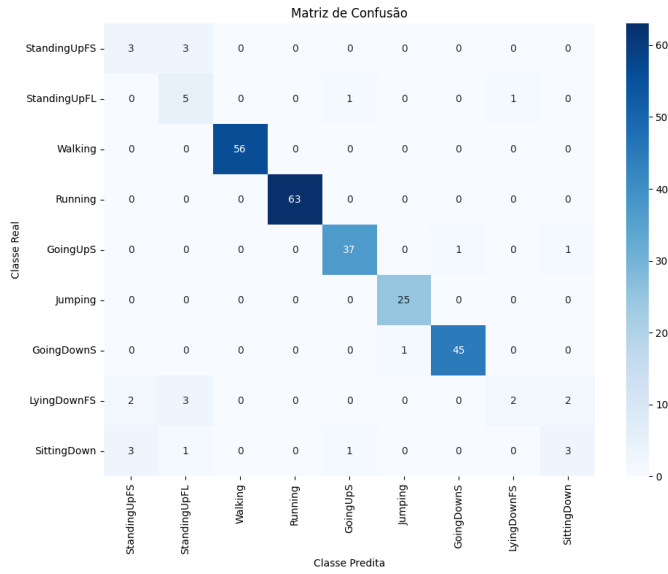


Fig. 4: Matriz de Confusão do Modelo

VIII. CONCLUSÃO

Diante dos resultados experimentais obtidos, conclui-se que a arquitetura de rede neural desenvolvida apresenta alta eficiência na tarefa de reconhecimento de padrões para cenários dinâmicos, mas demanda refinamentos estruturais na base de dados para garantir consistência preditiva global.

Embora o modelo atinja uma excelente acurácia global de 92,3% e uma baixa perda de 0,198, a análise minuciosa do relatório de classificação revela uma disparidade crítica: o desempenho é impecável (F1-Score entre 0,95 e 1,00) para atividades de locomoção como Walking, Running e subida/descida de escadas, mas despenca drasticamente (F1-Score entre 0,33 e 0,53) em estados estáticos ou de transição (LyingDownFS, SittingDown, StandingUp). Esta distorção ocorre devido ao severo desbalanceamento do dataset, visto que o suporte dessas classes minoritárias é excessivamente baixo (variando de apenas 6 a 9 amostras). Tal escassez amostral infla artificialmente a acurácia macro ponderada da rede, mascarando a incapacidade atual da LSTM de discriminar movimentos corporais sutis e posições paradas.

TABLE I: Relatório de Classificação

Classificação	Precisão	Recall	F1-Score	Support
StandingUpFS	0.38	0.50	0.43	6
StandingUpFL	0.42	0.71	0.53	7
Walking	1.00	1.00	1.00	56
Running	1.00	1.00	1.00	63
GoingUpS	0.95	0.95	0.95	39
Jumping	0.96	1.00	0.98	25
GoingDownS	0.98	0.98	0.98	46
LyingDownFS	0.67	0.22	0.33	9
SittingDown	0.50	0.38	0.43	8
Accuracy			0.92	259
Macro avg	0.76	0.75	0.74	259
Weighted avg	0.93	0.92	0.92	259

O Notebook do Colab deste trabalho está disponível na URL a seguir: [Nycksandro_Lima_Trabalho06](#)

REFERENCES

- [1] D. Micucci, M. Mobilio, and P. Napoletano, "Unimib shar: A dataset for human activity recognition using acceleration data from smartphones," *Applied Sciences*, vol. 7, no. 11, p. 1101, 2017.